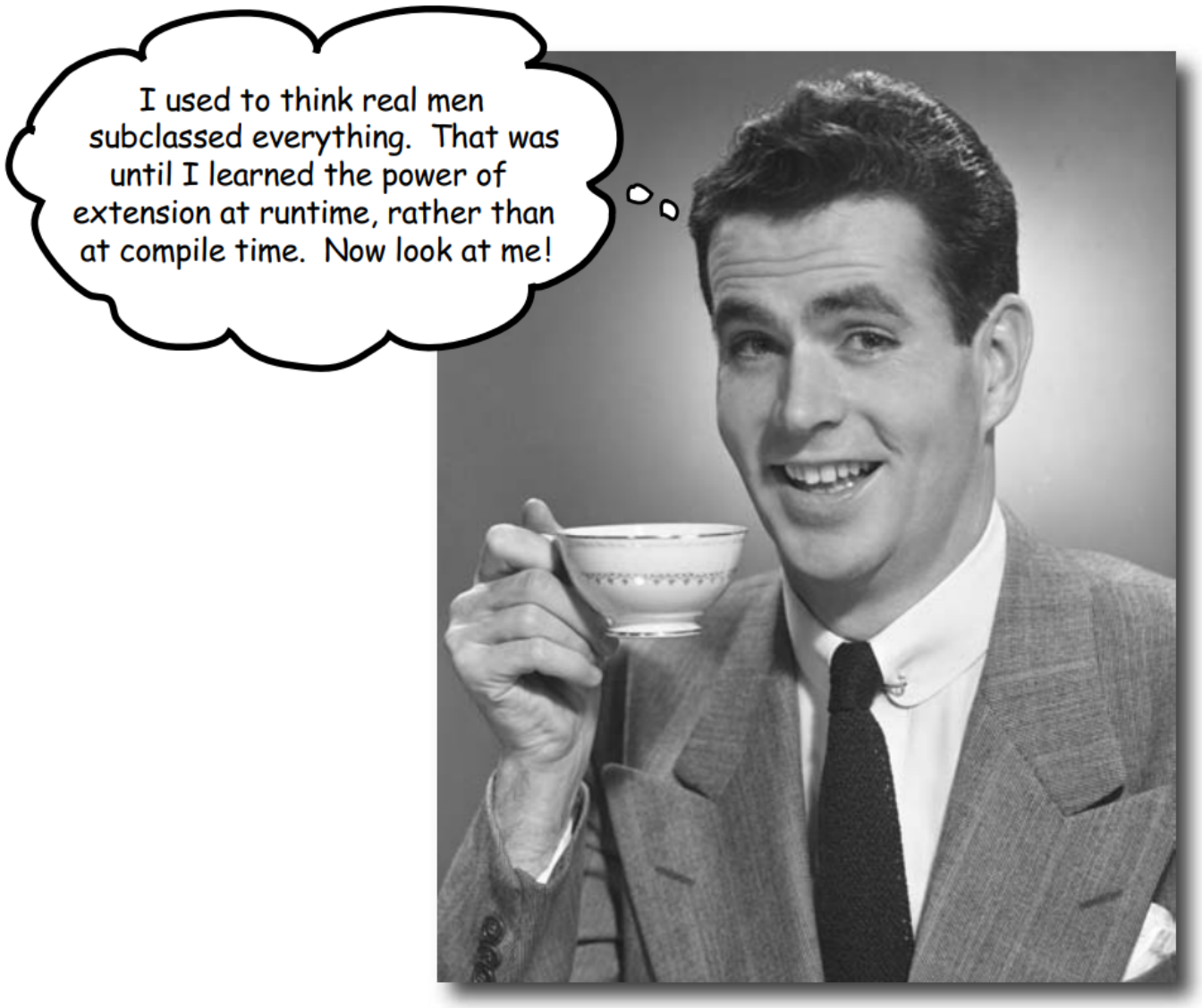
Chương 3: Decorator Pattern – Đối tượng trang trí



Decorator Pattern – Gọi chương này là "Thiết kế đôi mắt của bạn cho sự thừa kế"

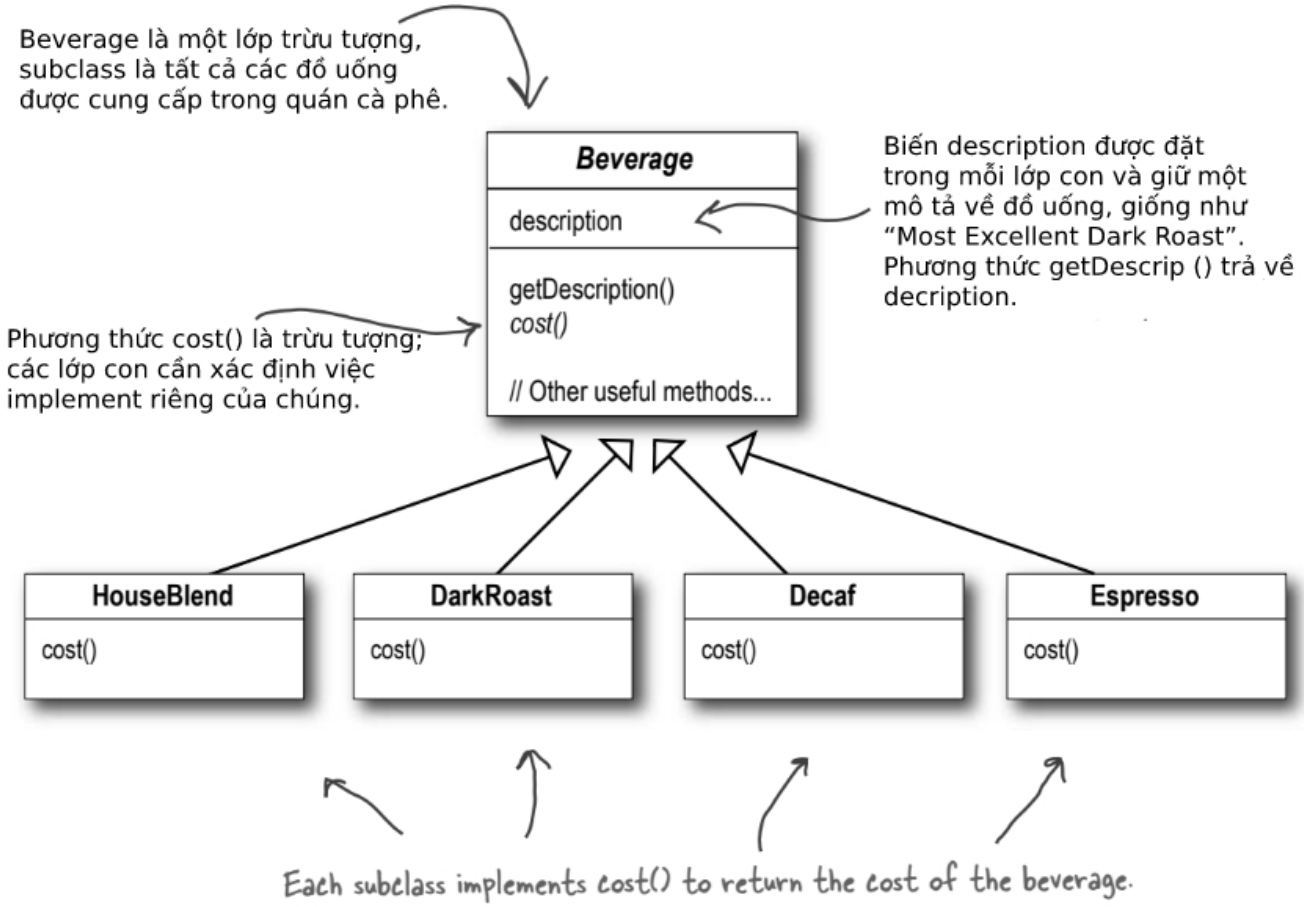
Chúng tôi sẽ kiểm tra lại việc lạm dụng thừa kế thường gặp và bạn sẽ học cách "trang trí" các lớp của mình khi runtime bằng cách sử dụng việc kết hợp (compose) các đối tượng. Tại sao à? Một khi bạn biết các kỹ thuật decorating (với Decorator Pattern), bạn sẽ có thể cung cấp cho các object của mình những hành vi mới mà không cần phải thay đổi code đối với các lớp bên trong.

Chào mừng đến với Starbuzz Coffee

Starbuzz Coffee là cửa hàng cà phê phát triển nhanh. Ở bất kì nơi nào đó, hãy nhìn qua đường; bạn sẽ thấy một tiệm Starbuzz Coffee hiện diện.

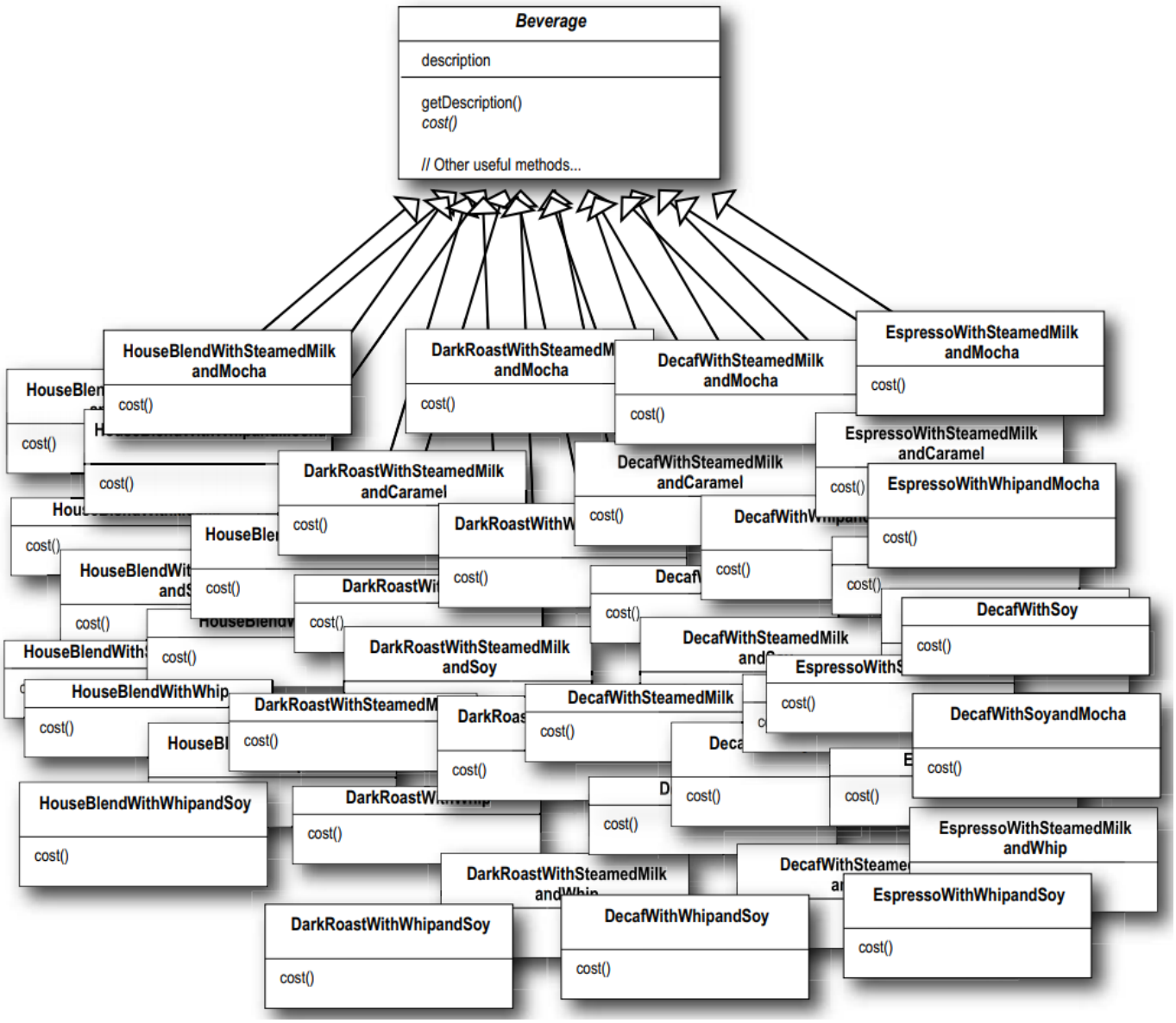
Bởi vì họ đã phát triển quá nhanh, họ phải cập nhật hệ thống đặt hàng để phù hợp với các dịch vụ đồ uống của mình.

Khi họ bắt đầu kinh doanh, họ đã thiết kế các lớp như thế này …



Ngoài ly cà phê tiêu chuẩn của bạn, bạn cũng có thể yêu cầu kèm thêm một số thứ khác như sữa, đậu nành và mocha, và tất cả đều được phủ lên trên bằng sữa tươi. Starbuzz tính phí cho mỗi thứ này, vì vậy họ thực sự cần phải xây dựng chúng trong hệ thống đặt hàng của mình.

Đây là nỗ lực đầu tiên của họ …



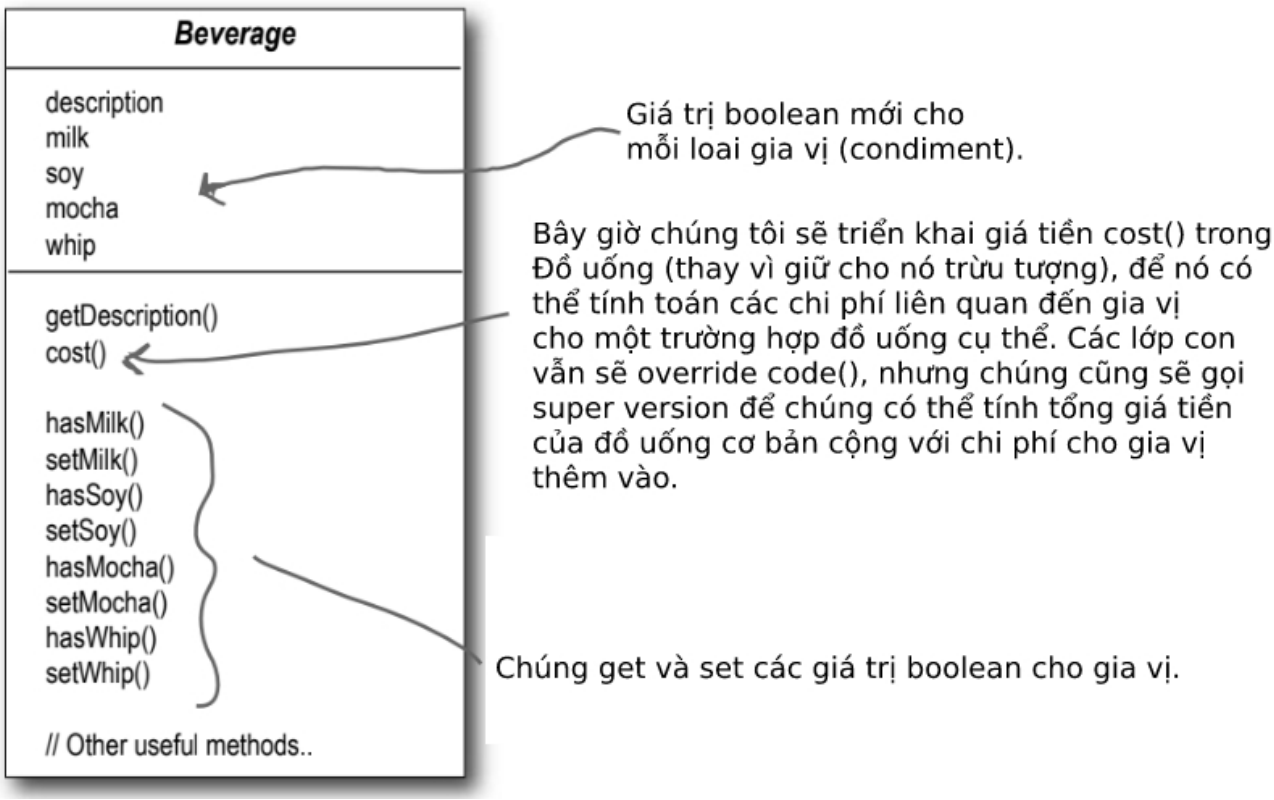
Sử dụng sức mạnh của bộ não

Khá rõ ràng rằng Starbuzz đã tạo ra một cơn ác mộng bảo trì cho chính họ. Điều gì xảy ra khi giá sữa tăng? Họ sẽ làm gì khi thêm một lớp phủ caramel mới?

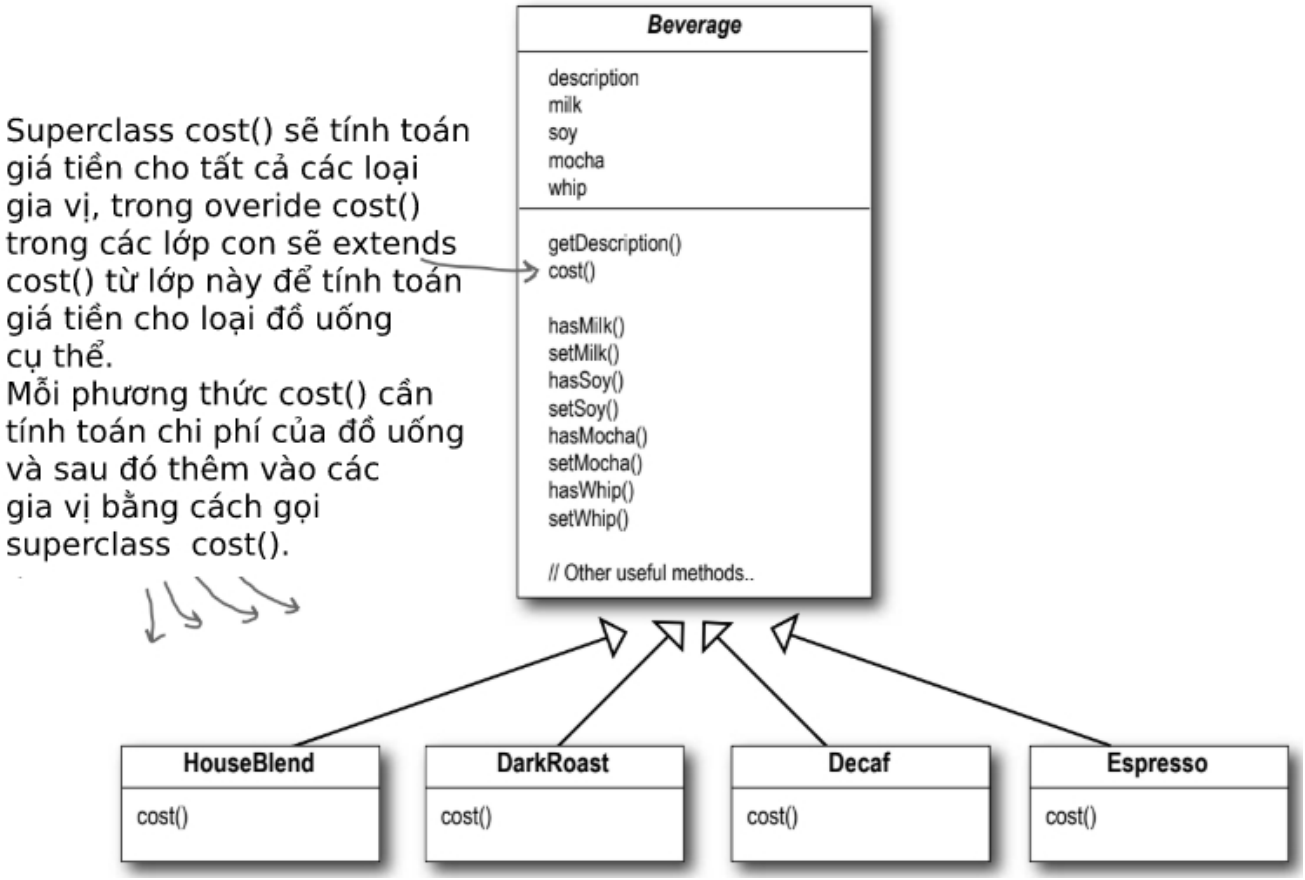
Bỏ qua vấn đề bảo trì, chúng có vi phạm những nguyên tắc thiết kế mà chúng tôi đã đề cập trước đây không?



Chà, hãy thử nghiệm một chút. Bắt đầu với base class Beverage (Đồ uống) và thêm các biến instance để thể hiện các loại toping đi kèm: milk, soy, mocha và whip…



Bây giờ, hãy thêm vào các lớp con, một cho mỗi loại đồ uống trên menu:



Sharpen your pencil

Viết các phương thức cost() cho các lớp sau:

```
public class Beverage {
   public double cost() {

   }
}
```

```
public class DarkRoast extends Beverage {

   public DarkRoast() {
      description = "Most Excellent Dark Roast";
   }

   public double cost() {

   }
}
```

Đáp án:

```
public class Beverage {

  // declare instance variables for milkCost,
  //  soyCost, mochaCost, and whipCost, and
  // getters and setters for milk, soy, mocha
  // and whip.

public double cost() {

      double condimentCost = 0.0;
      if (hasMilk()) {
         condimentCost += milkCost;
      }
      if (hasSoy()) {
         condimentCost += soyCost;
      }
      if (hasMocha()) {
         condimentCost += mochaCost;
      }
      if (hasWhip()) {
         condimentCost += whipCost;
      }
      return condimentCost;
   }
}
```

```
public class DarkRoast extends Beverage {

   public DarkRoast() {
      description = "Most Excellent Dark Roast";
   }

   public double cost() {

      return 1.99 + super.cost();


   }
}
```





Làm nhọn bút chì của bạn

Những yêu cầu hoặc các yếu tố khác có thể thay đổi sẽ ảnh hưởng đến thiết kế này là gì?

- Thay đổi giá các loại toping sẽ buộc họ phải thay đổi code hiện có.

- Toping mới sẽ buộc chúng ta thêm các phương thức mới và thay đổi cách tính chi phí cost() trong lớp cha.
- Chúng tôi có thể có đồ uống mới. Đối với một số đồ uống này (trà đá – Tea?), Toping có thể không phù hợp nữa, chúng ta không thể bỏ mocha vào trà đá, nhưng ở đây, lớp con Tea vẫn sẽ kế thừa các phương thức như hasWhip().
- Điều gì nếu một khách hàng muốn một lượng mocha gấp đôi?
- Đến lượt bạn viết thêm một số yếu tố nữa:.....

Sư phụ và học trò

Sư phụ: Grasshopper, đã một thời gian kể từ cuộc nói chuyện cuối cùng của chúng ta. Con đã từng suy nghĩ thật lâu về thừa kế chưa?

Học trò: Vâng, thưa thầy. Mặc dù tính kế thừa rất mạnh mẽ nhưng con đã học được rằng nó không phải lúc nào cũng dẫn đến các thiết kế linh hoạt nhất hoặc có thể bảo trì.

Sư phụ: À đúng rồi, con đã có một số tiến bộ. Vì vậy, hãy cho thầy biết Học trò của thầy, làm thế nào con sẽ đạt được "tái sử dụng" nếu không thông qua thừa kế?

Học trò: Sư phụ, con đã học được rằng có nhiều cách để kế thừa hành vi trong thời gian chạy thông qua composition (kết hợp hành vi) và delegation (ủy thác hành vi cho các lớp khác).

Sư phụ: Tiếp tục đi…

Học trò: Khi con kế thừa hành vi bằng cách phân lớp, hành vi đó được đặt cố định tại thời gian biên dịch. Ngoài ra, tất cả các lớp con phải kế thừa cùng một hành vi. Tuy nhiên, nếu con có thể mở rộng một hành vi của đối tượng thông qua composition, thì con có thể thực hiện điều này một cách linh hoạt khi chạy.

Sư phụ: Rất tốt, Grasshopper, con đang bắt đầu thấy sức mạnh của composition.

Học trò: Có, con có thể thêm nhiều hành vi mới cho các đối tượng thông qua kỹ thuật này, bao gồm cả những hành vi mà người thiết kế lớp cha thậm chí không nghĩ tới. Và, con không phải đụng chạm vào code của họ!

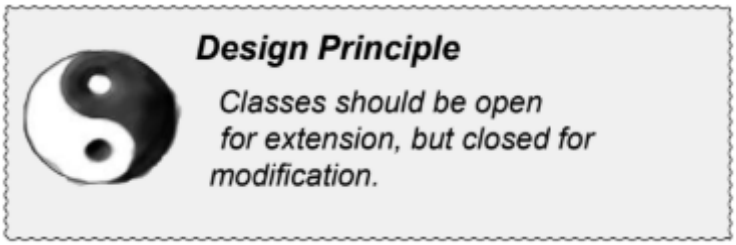
Sư phụ: Con đã học được gì về tác dụng của composition trong việc bảo trì code của mình?

Học trò: Vâng, đó là những gì con đã nhận được. Bằng cách kết hợp động các đối tượng, con có thể thêm chức năng mới bằng cách viết thêm code thay vì thay đổi code hiện có. Và vì không thay đổi code hiện tại, nên cơ hội xảy ra lỗi hoặc gây ra ảnh hưởng ngoài ý muốn trong code trước đó đã giảm đi nhiều.

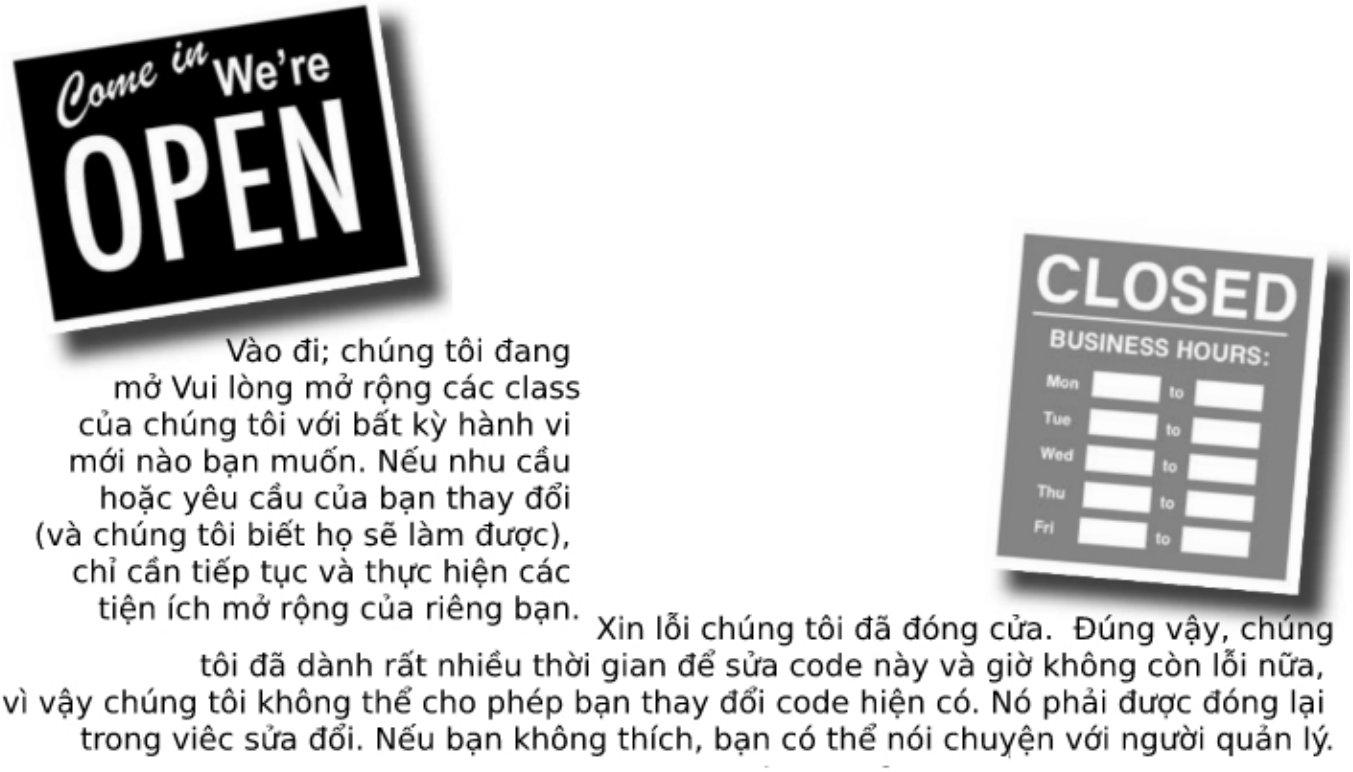
Sư phụ: Rất tốt. Đã đủ cho ngày hôm nay, Grasshopper. Thầy muốn con đi và suy ngẫm sâu hơn về chủ đề này … Hãy nhớ rằng, code nên được đóng (để thay đổi) như hoa sen vào buổi tối, nhưng mở (để mở rộng) như hoa sen vào buổi sáng.

Nguyên tắc đóng mở (The Open-Closed Principle)

Grasshopper đã nói về một trong những nguyên tắc thiết kế quan trọng nhất:



(Những class nên mở cho việc mở rộng, nhưng đóng cho việc sửa đổi)



Mục tiêu của chúng tôi là cho phép các lớp dễ dàng được mở rộng để kết hợp hành vi mới mà không cần sửa đổi code hiện có. Chúng ta nhận được gì nếu đạt được điều này? Các thiết kế có khả năng phục hồi để thay đổi và đủ khả thi để đảm nhận chức năng mới đáp ứng các yêu cầu thay đổi.

Không có câu hỏi ngớ ngẩn

Hỏi: Mở để mở rộng và đóng để sửa đổi? Nghe có vẻ rất mâu thuẫn. Làm thế nào một thiết kế có thể là cả hai?

Trả lời: Đó là một câu hỏi rất hay. Nó chắc chắn nghe có vẻ mâu thuẫn lúc đầu. Rốt cuộc, một cái gì đó càng ít sửa đổi, càng khó mở rộng, phải không? Mặc dù hóa ra, có một số kỹ thuật OO thông minh để cho phép các hệ thống được mở rộng, ngay cả khi chúng ta không thể thay đổi code cơ bản. Hãy suy nghĩ về Mẫu Người quan sát (https://toihocdesignpattern.com/head-first-design-patterns-tieng-viet-chuong-2-observer-pattern.html) (trong Chương 2 (https://toihocdesignpattern.com/head-first-design-patterns-tieng-viet-chuong-2-observer-pattern.html)) … bằng cách thêm Người quan sát (https://toihocdesignpattern.com/head-first-design-patterns-tieng-viet-chuong-2-observer-pattern.html) mới, chúng ta có thể mở rộng Subject bất cứ lúc nào mà không cần thêm code vào lớp Subject. Bạn sẽ thấy khá nhiều cách mở rộng hành vi với các kỹ thuật thiết kế OO khác.

Hỏi: Được rồi, tôi hiểu Observable (https://toihocdesignpattern.com/head-first-design-patterns-tieng-viet-chuong-2-observer-pattern.html), nhưng làm thế nào để tôi thiết kế một cái gì đó có thể mở rộng, nhưng đóng để sửa đổi?

Trả lời: Nhiều mẫu đã được chúng tôi thử nghiệm các thiết kế, chúng có thể bảo vệ code của bạn tránh bị sửa đổi bằng cách cung cấp các extension. Trong chương này, bạn sẽ thấy một ví dụ hay về việc sử dụng mẫu Trang trí (**Decorator Pattern**) để tuân theo nguyên tắc Đóng-Mở.

Hỏi: Làm thế nào tôi có thể làm cho **MỌI PHẦN** trong thiết kế của mình tuân theo Nguyên tắc Đóng mở?

Trả lời: Thông thường, bạn không thể. Làm cho thiết kế OO linh hoạt và mở để mở rộng mà không cần sửa đổi code hiện có cần có thời gian và công sức. Nói chung, chúng tôi không buộc tất cả các phần trong thiết kế của chúng tôi (và có lẽ sẽ lãng phí) tuân theo điều này. Tuân theo Nguyên tắc Đóng mở thường đưa ra các mức độ trừu tượng mới, và thêm sự phức tạp vào code. Chỉ nơi nào bạn muốn có nhiều khả năng thay đổi thì hãy áp dụng các nguyên tắc ở đó.

Hỏi: Làm thế nào để tôi biết nơi nào có nhiều khả năng thay đổi?

Trả lời: Đó là một phần vấn đề kinh nghiệm trong việc thiết kế các hệ thống OO và cũng là vấn đề sự hiểu biết lĩnh vực bạn đang làm việc. Nhìn vào các ví dụ khác sẽ giúp bạn tìm hiểu thêm những nơi hay thay đổi trong thiết kế của riêng bạn.

Mặc dù có vẻ như mâu thuẫn, nhưng có những kỹ thuật cho phép mở rộng code mà không cần sửa đổi trực tiếp.

Hãy cẩn thận khi chọn các nơi code cần được mở rộng; áp dụng Nguyên tắc đóng mở MỌI NƠI là lãng phí, không cần thiết và có thể dẫn đến code phức tạp, khó hiểu.



Đến với Decorator Pattern

Được rồi, chúng tôi đã thấy rằng việc đại diện cho đồ uống của chúng tôi cộng với kế hoạch tính giá các loại toping bằng sự kế thừa không có kết quả tốt – chúng tôi có các "vụ nổ" lớp, thiết kế cứng nhắc hoặc chúng tôi thêm các chức năng cho baseclass không phù hợp với một số lớp con.

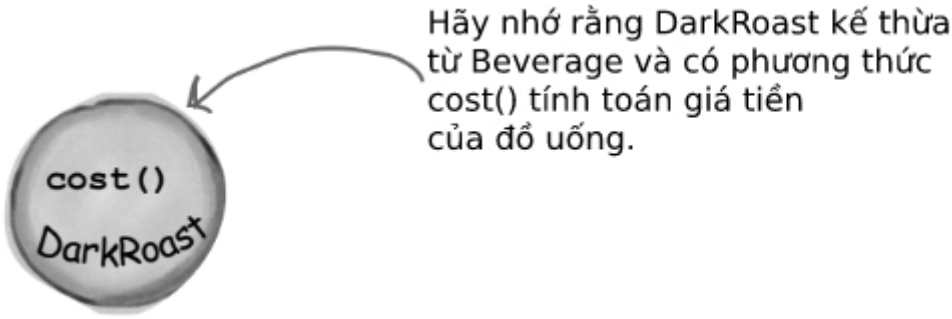
Vì vậy, ở đây, những gì chúng tôi sẽ làm thay vào đó: chúng tôi sẽ bắt đầu với một loại đồ uống và "trang trí" cho nó bằng các loại toping trong khi chạy (runtime). Ví dụ: nếu khách hàng muốn uống Dark Roast với toping là Mocha và Whip, thì chúng tôi sẽ:

1. Lấy một đối tượng DarkRoast
2. Trang trí nó với một đối tượng Mocha
3. Trang trí nó với một đối tượng Whip
4. Gọi phương thức `cost()` và dựa vào ủy quyền (delegate) để tính giá tiền cho các loại toping.

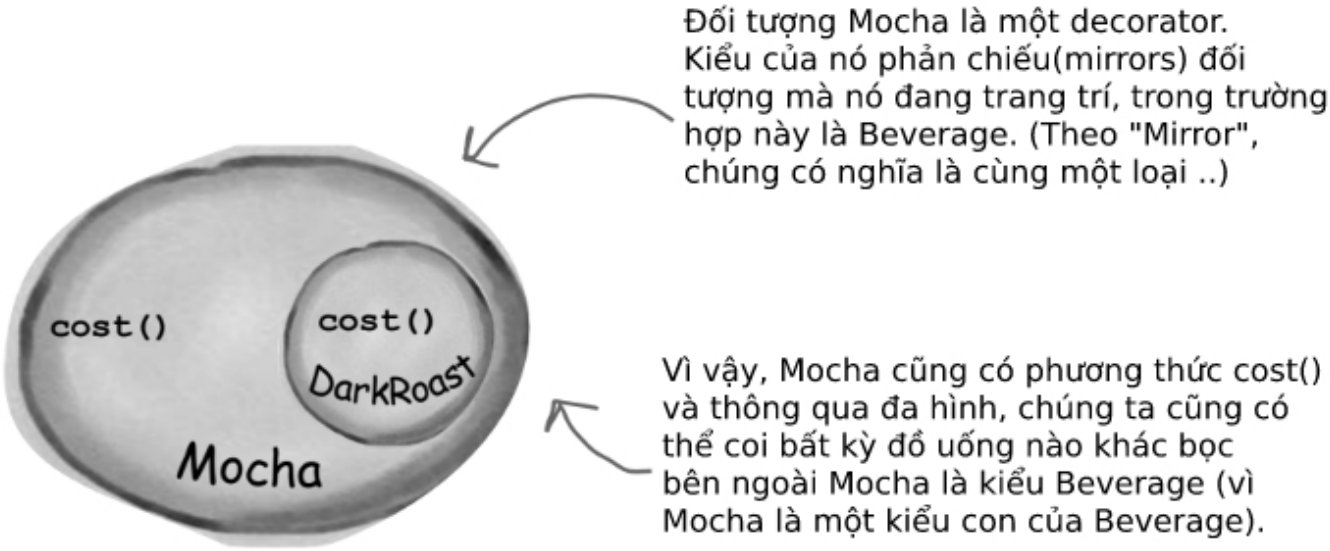
Được rồi, nhưng làm thế nào để bạn trang trí một đối tượng, và làm thế nào để delegation (ủy quyền)? **Một gợi ý:** hãy nghĩ về các đối tượng trang trí như các "trình bao bọc" (wrappers) thứ sẽ bao bên ngoài đối tượng chính của chúng ta.

Xây dựng một đơn đặt hàng đồ uống với Decorator Pattern

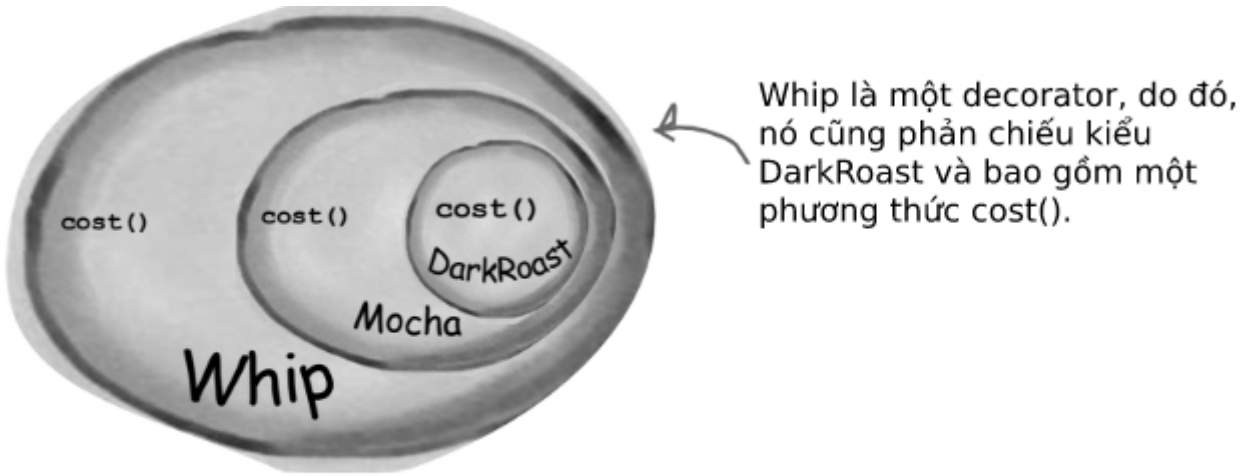
1. Chúng tôi bắt đầu với đối tượng **DarkRoast**.



2. Khách hàng muốn **Mocha**, vì vậy chúng tôi tạo một đối tượng **Mocha** và bọc nó xung quanh **DarkRoast**.

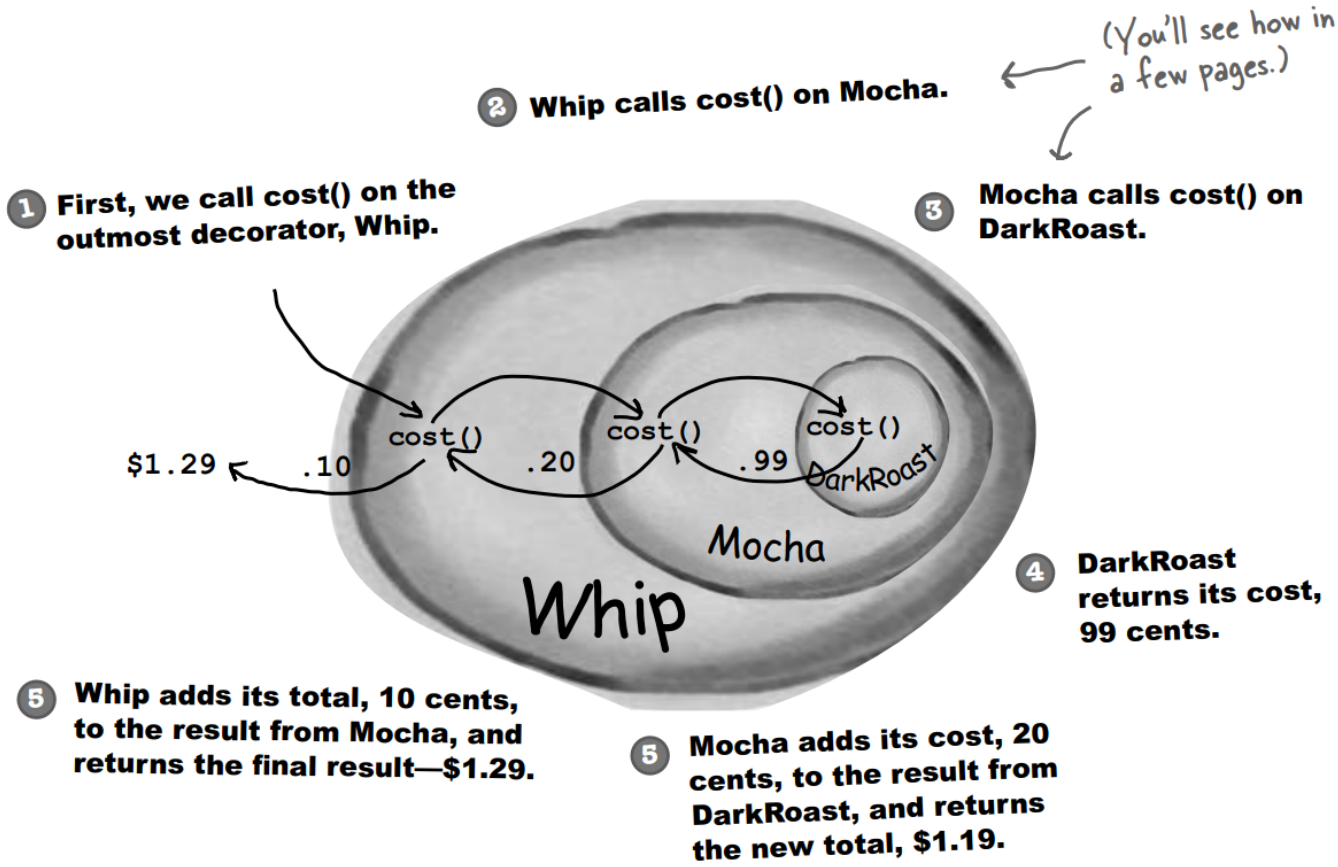


3. Khách hàng cũng muốn **Whip**, vì vậy chúng tôi tạo ra một trang trí **Whip** và bọc nó ngoài **Mocha**.



4. Bây giờ là thời gian để tính toán giá tiền ***cost()*** cho khách hàng. Chúng ta thực hiện điều này bằng cách gọi ***cost()*** trên decorator ngoài cùng là **Whip** và **Whip** sẽ ủy thác tính toán chi phí cho các đối tượng mà nó bọc bên trong. Một khi nó nhận được một giá tiền, nó sẽ cộng thêm vào giá tiền của Whip (hãy nghĩ

về đệ quy (https://techmaster.vn/posts/35006/khai-niem-de-quy-voi-java)).



ĐƯỢC RỒI, Ở ĐÂY, NHỮNG GÌ CHÚNG TA BIẾT CHO ĐẾN GIỜ...

- Các trình trang trí (Decorator) có kiểu giống như các đối tượng mà chúng trang trí (decorate).
- Bạn có thể sử dụng một hoặc nhiều trình trang trí (decorator) để bọc một đối tượng.
- Trình trang trí có kiểu giống như đối tượng mà nó trang trí, chúng ta có thể sử dụng một đối tượng được trang trí thay cho đối tượng ban đầu (được bọc).
- Decorator thêm hành vi của chính mình trước và/hoặc sau khi ủy thác cho đối tượng mà nó trang trí để thực hiện phần còn lại của công việc.
- Các đối tượng có thể được trang trí bất cứ lúc nào, vì vậy chúng ta có thể trang trí các đối tượng một cách linh hoạt trong runtime với nhiều trang trí tùy thích.

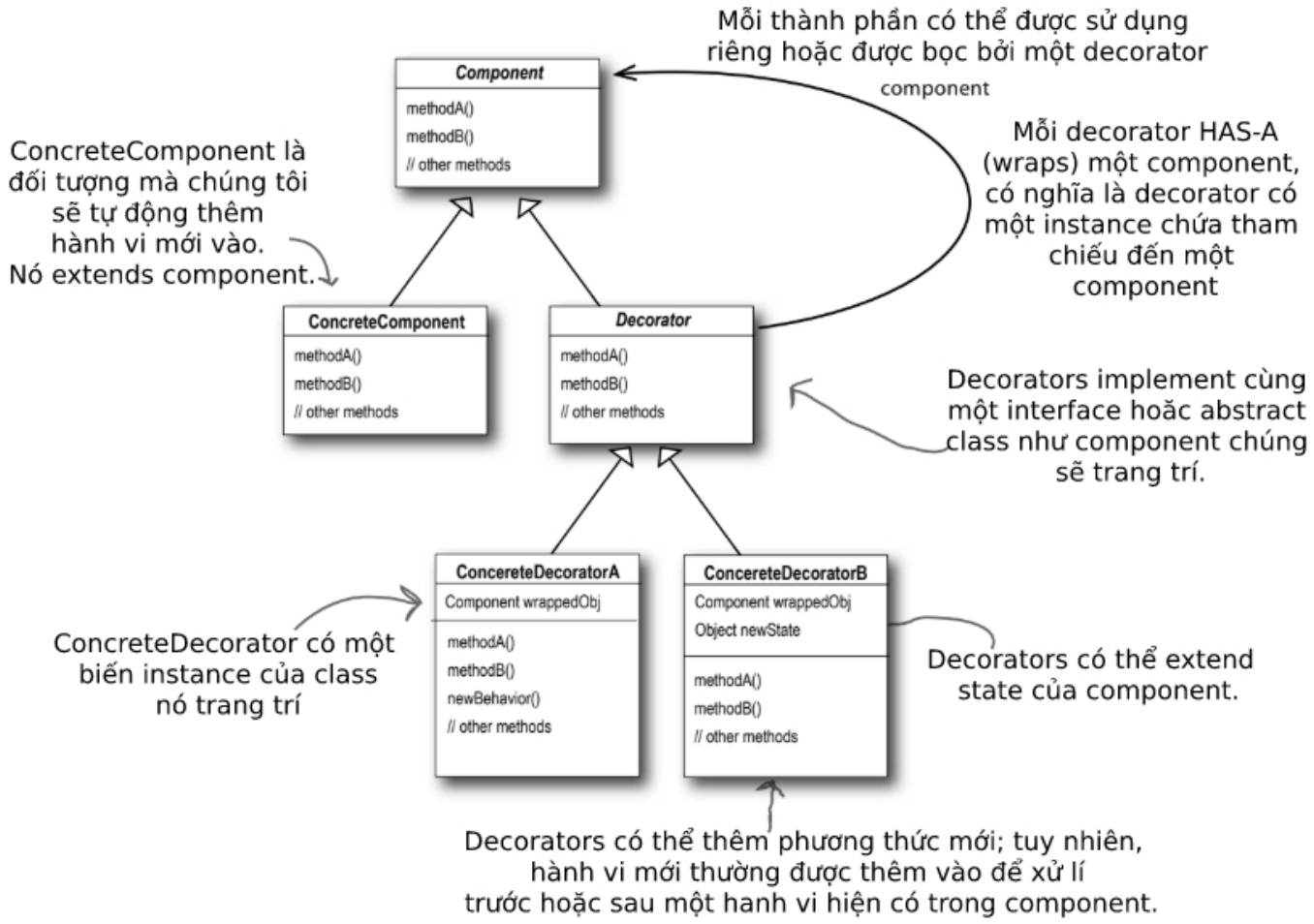
BÂY GIỜ HÃY XEM LÀM THẾ NÀO TẤT CẢ THỰC SỰ HOẠT ĐỘNG BẰNG CÁCH NHÌN VÀO ĐỊNH NGHĨA MẪU TRANG TRÍ VÀ VIẾT MỘT SỐ DÒNG CODE.

Định nghĩa Decorator Pattern

Trước tiên, hãy xem phần mô tả của Decorator Pattern

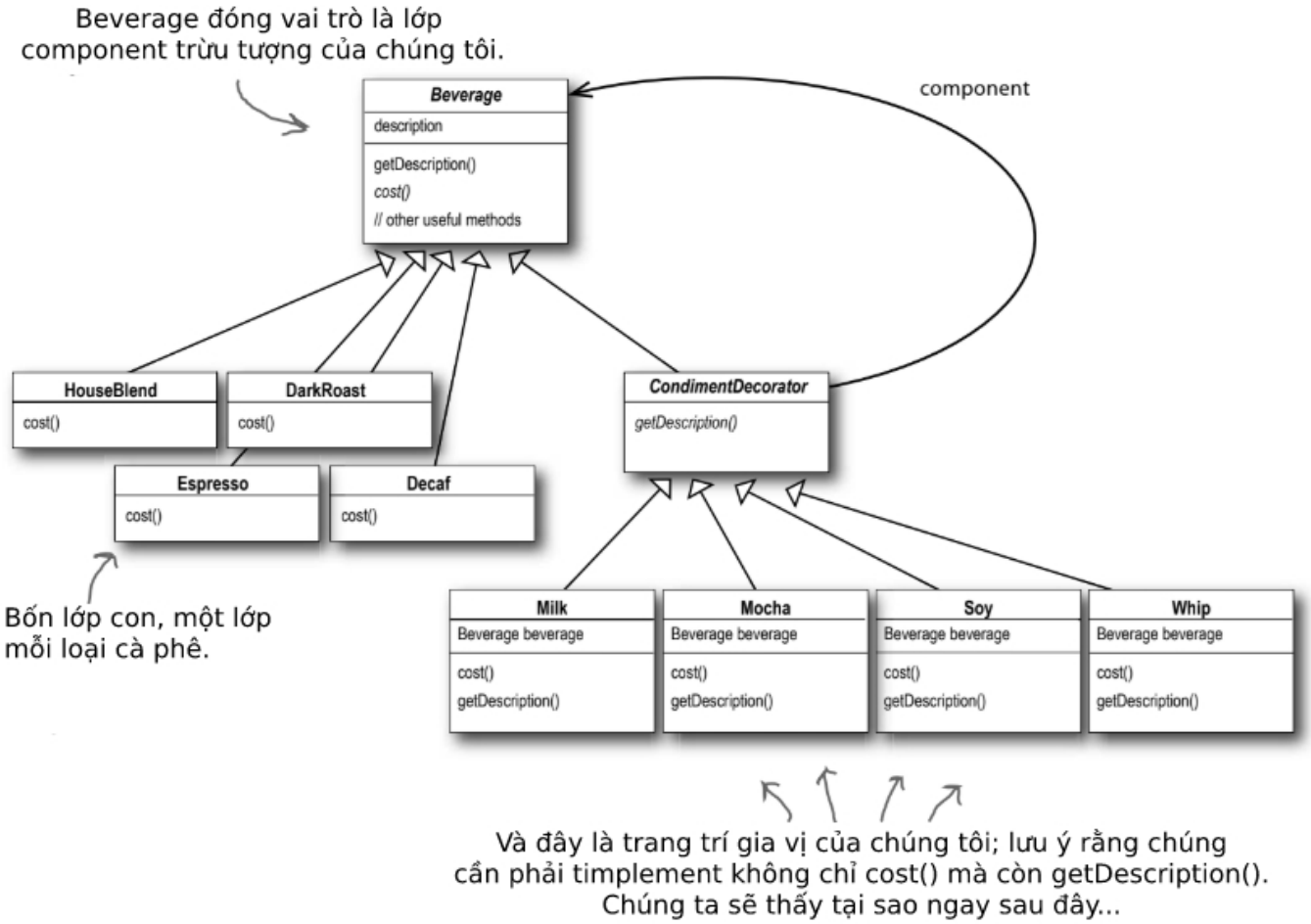
The Decorator Pattern attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

(Mẫu trang trí gắn các trách nhiệm bổ sung cho một đối tượng một cách linh hoạt. Mẫu trang trí cung cấp một sự thay thế linh hoạt cho phân lớp để mở rộng chức năng)



Trang trí lớp Beverages (đồ uống) của chúng ta

Được rồi, hãy để chúng tôi đưa đồ uống của Starbuzz vào mẫu này …



Sử dụng sức mạnh bộ não

Trước khi tiếp tục, hãy suy nghĩ về cách bạn implements phương thức cost() của cà phê và toping. Ngoài ra, hãy suy nghĩ về cách bạn implements phương thức getDescription() trong từng loại toping.

Đoạn hội thoại về Decorator Pattern

Một số nhầm lẫn về Kế thừa (Inheritance) so với Thành phần (Composition)

Mary: Được rồi, tôi có một chút bối rối … Tôi nghĩ rằng chúng ta sẽ không sử dụng sự kế thừa trong mô hình này, mà thay vào đó chúng ta sẽ dựa vào thành phần để thay thế.

Sue: Ý bạn là gì?

Mary: Nhìn vào sơ đồ lớp. Lớp **CondimentDecorator** đang ***extends*** lớp **Beverage**. Đó là sự kế thừa, phải không?

Sue: Đúng. Tôi nghĩ vấn đề quan trọng là các decorator có cùng loại với các đối tượng mà nó sẽ trang trí. Vì vậy, ở đây chúng tôi sử dụng tính kế thừa để đạt được kiểu giống nhau, nhưng chúng tôi không sử dụng tính kế thừa để có các hành vi.

Mary: Được rồi, tôi có thể thấy các trình trang trí decorator cần kiểu interface tương tự như các thành phần nó bọc bên trong bởi vì nó cần phải thay thế các thành phần đó. Nhưng còn hành vi đến từ đâu khi chúng không thừa kế từ lớp cha?

Sue: Khi chúng tôi kết hợp một **decorator** với **component**, chúng tôi sẽ thêm hành vi mới. Chúng ta đang có được hành vi mới không phải bằng cách kế thừa nó từ một lớp cha, mà bằng cách kết hợp các đối tượng lại với nhau.

Mary: Được rồi, vì vậy chúng tôi đang phân lớp lớp trừu tượng Beverage để có được loại chính xác, không kế thừa hành vi của nó. Các hành vi xuất hiện thông qua việc kết hợp các thành phần của decorators (toping) với các base components (đồ uống) cũng như các decorator (đồ uống) khác.

Mary: Ooooh, tôi hiểu rồi. Và bởi vì chúng tôi đang sử dụng việc kết hợp đối tượng, chúng tôi có được sự linh hoạt hơn rất nhiều về cách pha trộn và kết hợp toping và đồ uống. Rất tuyệt vời.

Sue: Có, nếu chúng tôi dựa vào thừa kế, thì hành vi của chúng tôi chỉ có thể bị cố định tại thời gian biên dịch. Nói cách khác, chúng ta chỉ có được bất kỳ hành vi nào mà superclass mang lại cho chúng ta hoặc chúng ta override. Với composition, chúng ta có thể kết hợp các trang trí theo bất kỳ cách nào chúng ta thích … in runtime.

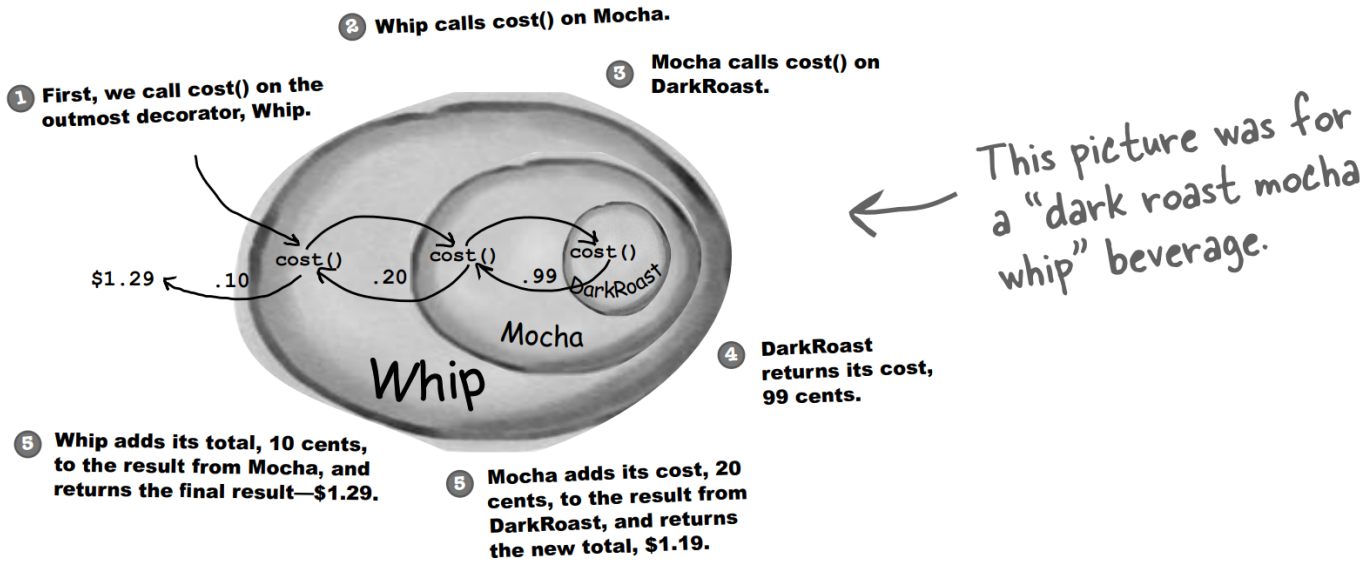
Mary: Và theo tôi hiểu, chúng ta có thể implement một decorators mới bất cứ lúc nào để thêm hành vi mới. Nếu chúng tôi dựa vào sự kế thừa, chúng tôi phải thay đổi code hiện có bất cứ khi nào chúng tôi muốn hành vi mới.

Sue: Chính xác.

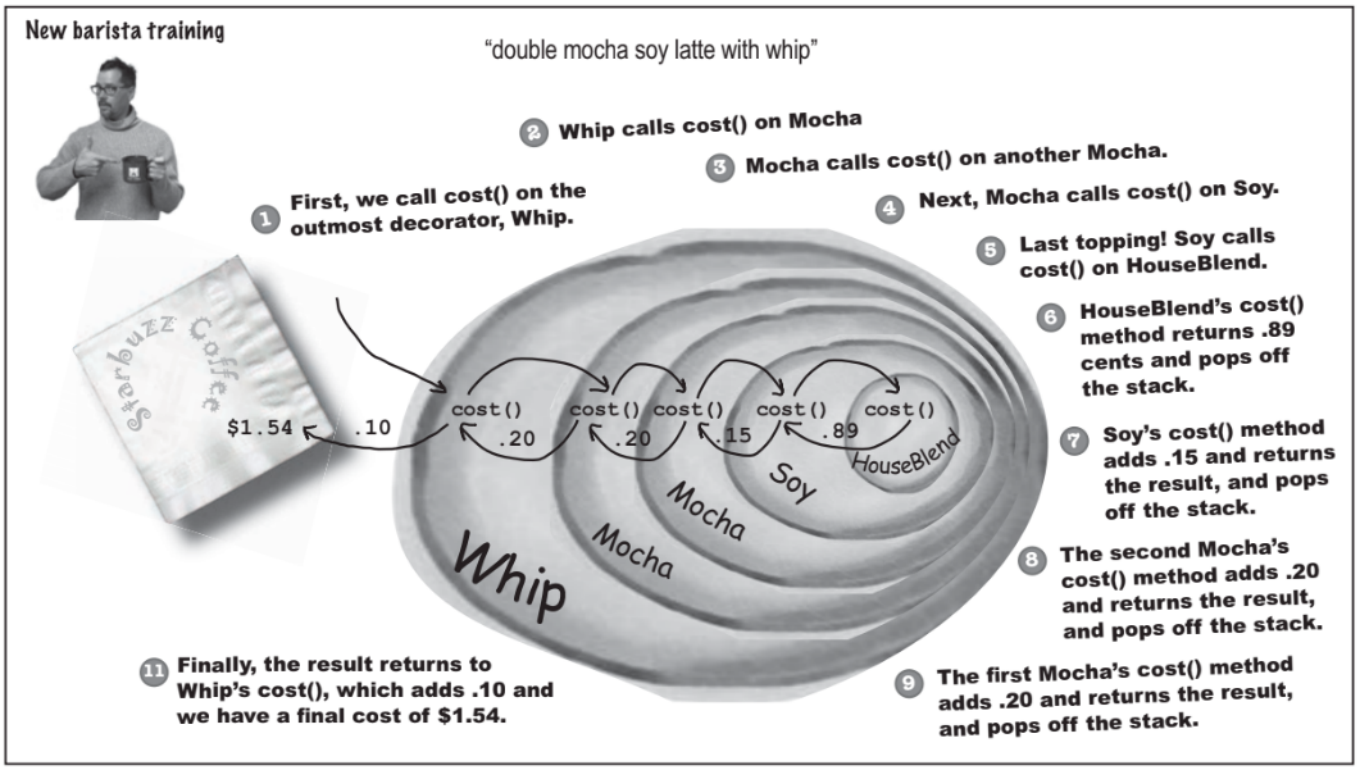
Mary: Một câu hỏi nữa. Nếu tất cả những gì chúng ta cần kế thừa là loại component, tại sao chúng ta không sử dụng interface thay vì một lớp abstract cho lớp Beverage?

Sue: Chà, hãy nhớ rằng, khi chúng tôi nhận được code, Starbuzz đã có một lớp abstract Beverage. Theo quy tắc, Decorator Pattern sẽ chỉ định một abstract class, nhưng trong Java, rõ ràng, chúng ta có thể sử dụng một interface. Nhưng chúng tôi luôn cố gắng tránh thay đổi code hiện có, vì vậy đừng sửa nó nếu lớp trừu tượng vẫn hoạt động tốt.

Tạo một bức tranh cho những gì xảy ra khi đơn đặt hàng cho một ly cà phê với toping là soy, gấp đôi mocha, và whip. Sử dụng menu để có được giá chính xác và vẽ hình ảnh của bạn bằng cách sử dụng cùng định dạng chúng tôi đã sử dụng dưới đây:



Hãy suy nghĩ trước khi xem đáp án:



Viết Code cho Starbuzz

Đã đến lúc đưa thiết kế này vào một đoạn code thực.

Hãy bắt đầu với lớp Beverage, mà không cần phải thay đổi từ thiết kế ban đầu của Starbuzz. Hãy xem, hãy xem:

```
public abstract class Beverage {
    String description = "Unknown Beverage";

    public String getDescription() {
        return description;
    }

    public abstract double cost();
}
```

Beverage is an abstract class with the two methods getDescription() and cost().

getDescription is already implemented for us, but we need to implement cost() in the subclasses.

Beverage rất đơn giản. Chúng ta cũng hãy tạo lớp Condiments (đây là lớp cha của mọi toping – là một Decorator) implements Beverage:

First, we need to be interchangeable with a Beverage, so we extend the Beverage class.

```
public abstract class CondimentDecorator extends Beverage {
    public abstract String getDescription();
}
```

We're also going to require that the condiment decorators all reimplement the getDescription() method. Again, we'll see why in a sec...

Code cho các lớp đồ uống

Bây giờ, chúng tôi đã đưa các lớp cơ sở của mình ra khỏi đường đi, hãy để chúng implements lớp Beverage. Chúng tôi sẽ bắt đầu với Espresso. Hãy nhớ rằng, chúng ta cần thiết lập một description cho đồ uống cụ thể và cũng implements phương thức cost().

First we extend the Beverage class, since this is a beverage.

```
public class Espresso extends Beverage {

    public Espresso() {
        description = "Espresso";
    }

    public double cost() {
        return 1.99;
    }
}
```

To take care of the description, we set this in the constructor for the class. Remember the description instance variable is inherited from Beverage.

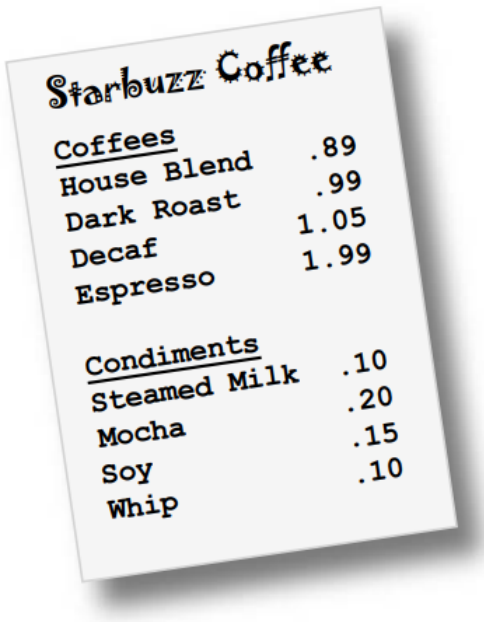
Finally, we need to compute the cost of an Espresso. We don't need to worry about adding in condiments in this class, we just need to return the price of an Espresso: $1.99.

```
public class HouseBlend extends Beverage {
    public HouseBlend() {
        description = "House Blend Coffee";
    }

    public double cost() {
        return .89;
    }
}
```

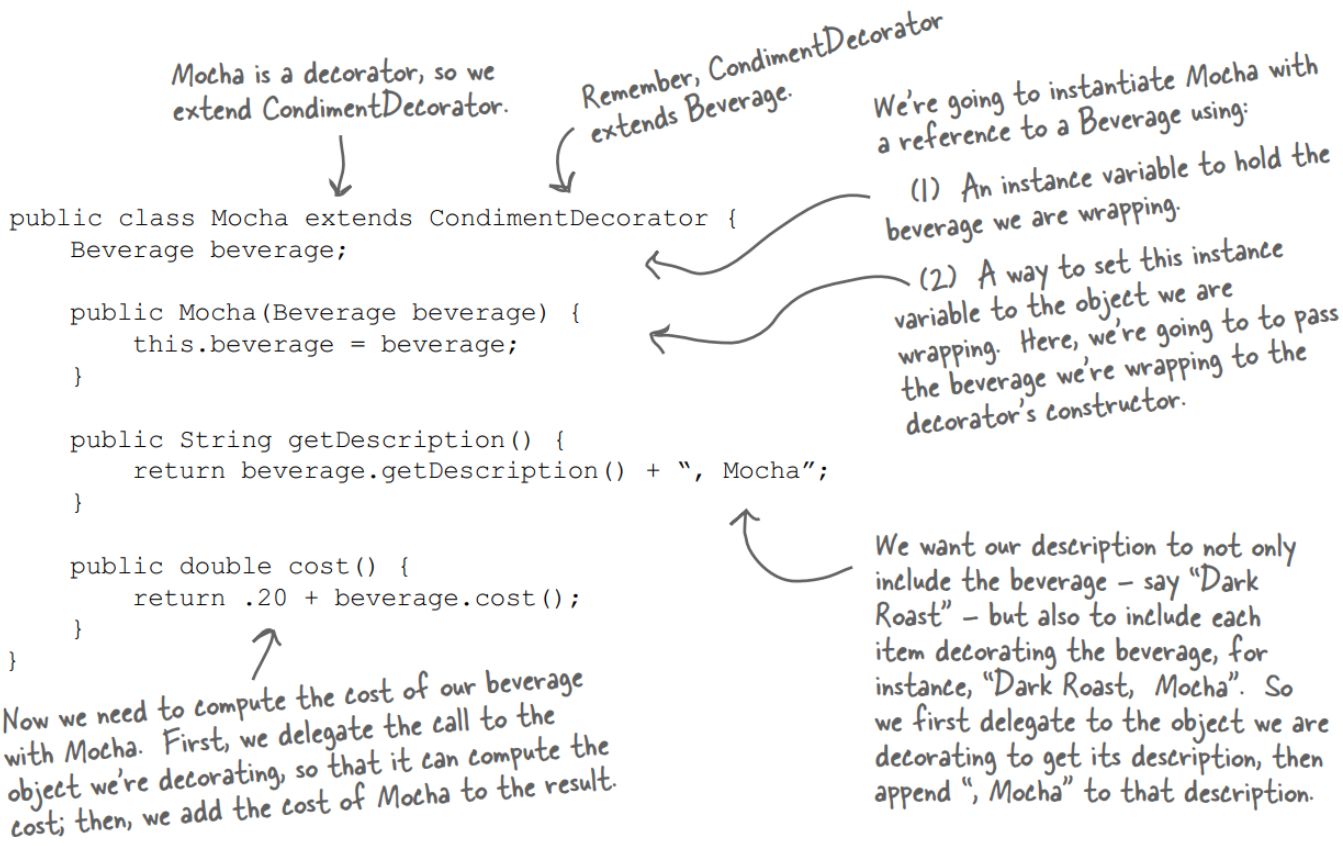
Okay, here's another Beverage. All we do is set the appropriate description, "House Blend Coffee," and then return the correct cost: 89¢.

You can create the other two Beverage classses (DarkRoast and Decaf) in exactly the same way.



Code các lớp toping (condiments)

Nếu bạn nhìn lại sơ đồ lớp Mẫu trang trí, bạn sẽ thấy chúng ta đã viết thành phần trừu tượng (Đồ uống), chúng ta có các thành phần cụ thể (HouseBlend) và chúng ta có lớp trừu tượng decorator (CondimentDecorator). Bây giờ nó thời gian để implements các loại toping. Dưới đây là **Mocha**:



On the next page we'll actually instantiate the beverage and wrap it with all its condiments (decorators), but first...

Phục vụ một ít cà phê

Xin chúc mừng. Đó là thời gian để ngồi lại, đặt một vài tách cà phê và ngạc nhiên trước thiết kế linh hoạt mà bạn đã tạo ra với Mẫu trang trí.

Dưới đây là một đoạn test code để order một tách cà phê

```
public class StarbuzzCoffee {

    public static void main(String args[]) {
        Beverage beverage = new Espresso();
        System.out.println(beverage.getDescription()
                + " $" + beverage.cost());

        Beverage beverage2 = new DarkRoast();
        beverage2 = new Mocha(beverage2);
        beverage2 = new Mocha(beverage2);
        beverage2 = new Whip(beverage2);
        System.out.println(beverage2.getDescription()
                + " $" + beverage2.cost());

        Beverage beverage3 = new HouseBlend();
        beverage3 = new Soy(beverage3);
        beverage3 = new Mocha(beverage3);
        beverage3 = new Whip(beverage3);
        System.out.println(beverage3.getDescription()
                + " $" + beverage3.cost());
    }
}
```

Order up an espresso, no condiments and print its description and cost.

Make a DarkRoast object.

Wrap it with a Mocha.

Wrap it in a second Mocha.

Wrap it in a Whip.

Finally, give us a HouseBlend with Soy, Mocha, and Whip.

* We're going to see a much better way of creating decorated objects when we cover the Factory Pattern (and the Builder Pattern, which is covered in the appendix).

Bây giờ chúng ta sẽ thấy order khi chạy

```
File Edit Window Help CloudsInMyCoffee
% java StarbuzzCoffee
Espresso $1.99
Dark Roast Coffee, Mocha, Mocha, Whip $1.49
House Blend Coffee, Soy, Mocha, Whip $1.34
%
```

Không có câu hỏi ngớ ngẩn

Hỏi: Tôi có một chút lo lắng về code khi test một lớp con của đồ uống, giả sử HouseBlend – và làm một cái gì đó, như phát hành mã giảm giá. Khi tôi đã bọc **HouseBlend** bằng các decorator, điều này sẽ không còn hiệu quả nữa.

Trả lời: Điều đó hoàn toàn chính xác. Nếu bạn có code dựa trên loại concrete component, decorators sẽ phá vỡ code đó. Miễn là bạn viết code theo kiểu abstract component, việc sử dụng các decorators sẽ "trong suốt" đối với code của bạn. Tuy nhiên, một khi bạn bắt đầu viết code chống lại các concrete component, bạn sẽ muốn suy nghĩ lại thiết kế ứng dụng của bạn và sử dụng trang trí của bạn.

Hỏi: Không dễ để một để một loại đồ uống của ai đó kết thúc với một decorator mà không phải là decorator ngoài cùng? Giống như nếu tôi có **DarkRoast** với **Mocha**, **Soy** và **Whip**, thật dễ dàng để viết code mà bằng cách nào đó kết thúc bằng một tham chiếu đến Soy thay vì Whip, có nghĩa là nó sẽ không bao gồm Whip trong đơn hàng.

Trả lời: Bạn chắc chắn có thể lập luận rằng bạn phải quản lý nhiều đối tượng hơn với Decorator Pattern và do đó, có nhiều khả năng coding errors sẽ đưa ra các vấn đề như bạn nói. Tuy nhiên, decorator thường được tạo bằng cách sử dụng các mẫu khác như **Factory** và **Builder**. Một khi chúng tôi đã trình bày các mẫu này, bạn sẽ thấy rằng việc tạo ra concrete component với decorator của nó là một gói được đóng gói tốt và không dẫn đến các loại vấn đề bên trên.

Hỏi: Decorator có thể biết về các Decorator khác trong chuỗi không? tôi muốn phương thức ***getDecription()*** của tôi in dòng chữ "***Whip, Double Mocha***" thay vì của "***Mocha, Whip, Mocha***"? Điều đó đòi hỏi decorator ngoài cùng của tôi biết tất cả những decorator mà nó đang bọc.

Trả lời: Decorator có nghĩa là thêm hành vi cho đối tượng nó bọc. Khi bạn cần biết đến nhiều lớp trong chuỗi decorator, bạn đang bắt đầu đưa decorator vượt ra ngoài ý định thực sự của nó. Tuy nhiên, những điều như vậy là có thể. Hãy tưởng tượng một decorator **CondimentPrettyPrint** phân tích cú pháp lớp cuối cùng và có thể in ra "***Mocha, Whip, Mocha***" thành "***Whip, Double Mocha***".

Bài tập:

Bạn bè của chúng tôi tại Starbuzz đã giới thiệu "size" vào menu của họ. Bây giờ bạn có thể đặt một tách cà phê với kích thước **tall**, **grande** và **venti** (nhỏ, vừa và lớn). Starbuzz thấy đây là một phần của lớp cà phê, vì vậy họ đã thêm hai phương thức vào lớp Beverage: ***setSize()*** và ***getSize()***. Họ cũng thích các loại gia vị được tính theo kích cỡ, vì vậy, ví dụ, Soy có giá lần lượt là 10¢, 15¢ và 20¢ cho các loại cà phê **tall**, **grande** và **venti**.

Làm thế nào bạn sẽ thay đổi các lớp decorator để xử lý sự thay đổi trong các yêu cầu?

Đáp án:

```
public class Soy extends CondimentDecorator {
    Beverage beverage;

    public Soy(Beverage beverage) {
        this.beverage = beverage;
    }

    public int getSize() {
        return beverage.getSize();
    }

    public String getDescription() {
        return beverage.getDescription() + ", Soy";
    }

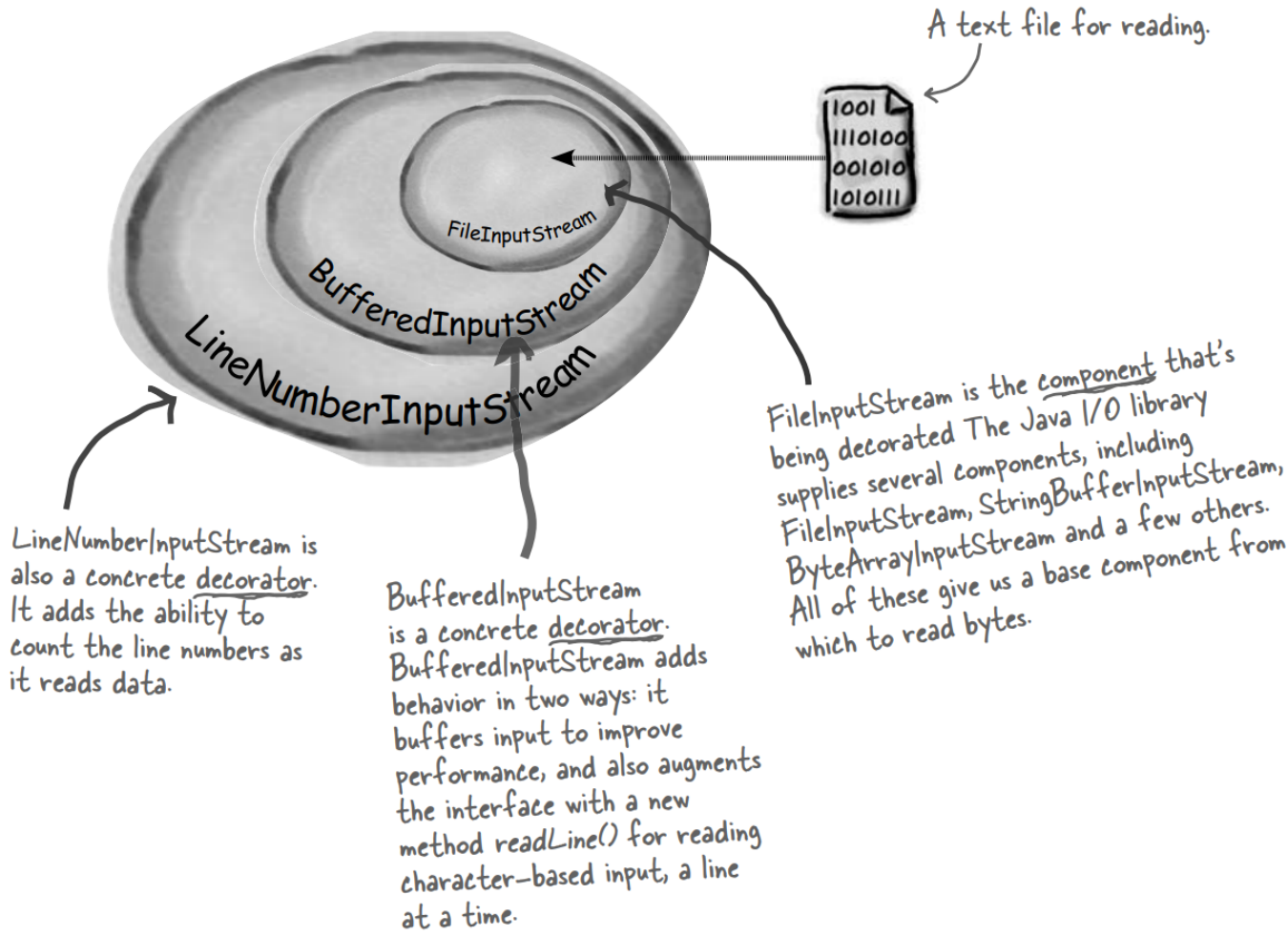
    public double cost() {
        double cost = beverage.cost();
        if (getSize() == Beverage.TALL) {
            cost += .10;
        } else if (getSize() == Beverage.GRANDE) {
            cost += .15;
        } else if (getSize() == Beverage.VENTI) {
            cost += .20;
        }
        return cost;
    }
}
```

Now we need to propagate the getSize() method to the wrapped beverage. We should also move this method to the abstract class since it's used in all condiment decorators.

Here we get the size (which propagates all the way to the concrete beverage) and then add the appropriate cost.

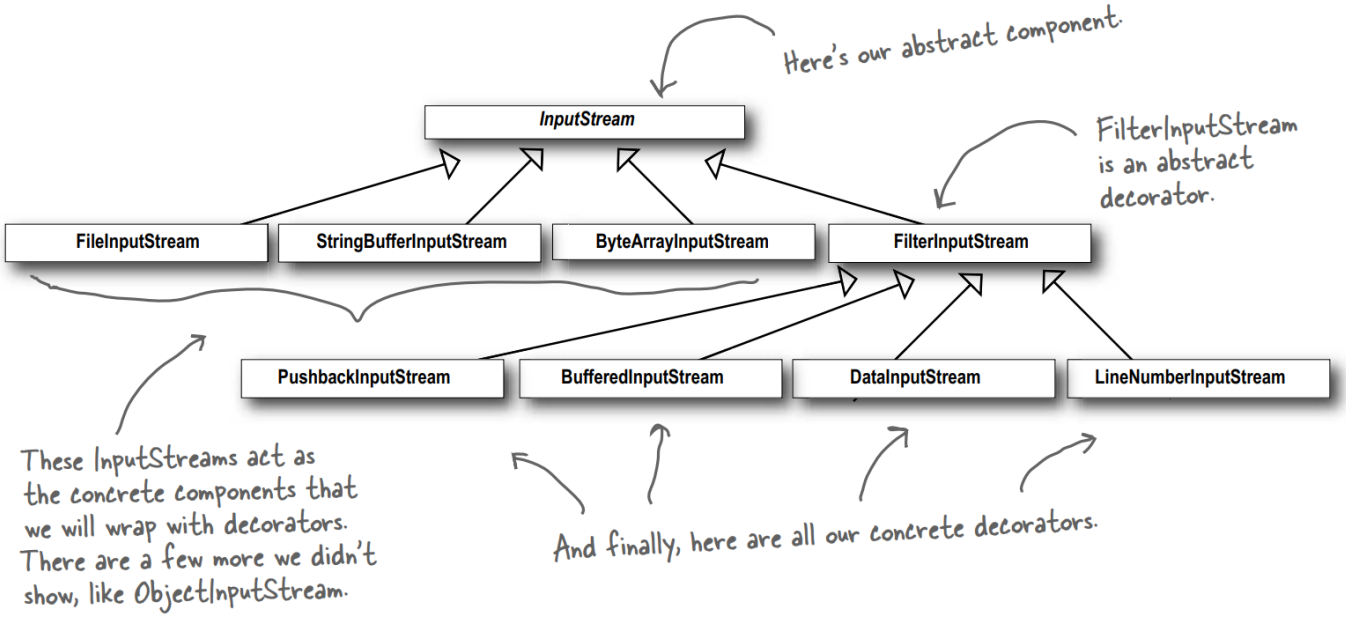
Thế giới thực của Decorator Pattern: Java I/O

Số lượng lớn các lớp trong gói **java.io** là … quá nhiều. Đừng cảm thấy cô đơn nếu bạn nói rằng "whoa" một lần (và hai lần và ba lần) bạn đã xem API này. Nhưng bây giờ bạn đã biết Decorator Pattern, các lớp I/O sẽ có ý nghĩa hơn vì gói **java.io** chủ yếu dựa trên Decorator Pattern. Dưới đây là một bộ đối tượng điển hình sử dụng Decorator Pattern để thêm chức năng đọc dữ liệu từ một tệp:



Cả **Buffered**InputStream và **LineNumber**InputStream đều extends **Filter**InputStream – hoạt động như lớp trang trí trừu tượng.

DECORATING THE JAVA.IO CLASSES



Bạn có thể thấy rằng sơ đồ này không quá khác biệt so với thiết kế của Starbuzz. Bây giờ bạn sẽ ở một vị trí tốt để xem qua các tài liệu API java.io và soạn thảo các compose decorator trên các inputstream khác nhau.

Và bạn sẽ thấy rằng các *output stream* có cùng thiết kế. Và bạn có lẽ đã phát hiện ra rằng các luồng Reader/Writer (đối với dữ liệu dựa trên ký tự) phản ánh chặt chẽ thiết kế của các lớp luồng (có một chút khác biệt, nhưng đủ để tìm hiểu điều gì xảy ra).

Nhưng Java I/O cũng chỉ ra một trong những nhược điểm của Decorator Pattern: các thiết kế sử dụng mẫu này thường dẫn đến một số lượng lớn các lớp nhỏ có thể làm cho nhà phát triển đang cố gắng sử dụng API dựa trên Decorator cảm thấy choáng ngộp. Nhưng bây giờ bạn đã biết cách Decorator hoạt động, bạn có thể giữ mọi thứ trong khuôn mẫu và khi bạn đang sử dụng một API Decorator khổng lồ của người khác, bạn có thể tìm hiểu cách tổ chức các lớp của họ để có thể dễ dàng sử dụng gói để có được hành vi sau đó.

VIẾT TRÌNH TRANG TRÍ I/O JAVA CỦA RIÊNG BẠN

Được rồi, bạn biết Decorator Pattern, bạn đã thấy sơ đồ lớp I/O. Bạn nên sẵn sàng để viết input decorator của riêng bạn.

Làm thế nào: viết một **Decorator** chuyển đổi tất cả các ký tự chữ hoa thành chữ thường trong input stream. Nói cách khác, nếu chúng ta đọc "*I know the Decorator Pattern therefore I RULE!*" khi đó decorator của bạn sẽ chuyển thành "*i know the decorator pattern therefore i rule!*"

Don't forget to import java.io... (not shown)

First, extend the FilterInputStream, the abstract decorator for all InputStreams.

```
public class LowerCaseInputStream extends FilterInputStream {
    public LowerCaseInputStream(InputStream in) {
        super(in);
    }

    public int read() throws IOException {
        int c = super.read();
        return (c == -1 ? c : Character.toLowerCase((char)c));
    }

    public int read(byte[] b, int offset, int len) throws IOException {
        int result = super.read(b, offset, len);
        for (int i = offset; i < offset+result; i++) {
            b[i] = (byte)Character.toLowerCase((char)b[i]);
        }
        return result;
    }
}
```

Now we need to implement two read methods. They take a byte (or an array of bytes) and convert each byte (that represents a character) to lowercase if it's an uppercase character.

REMEMBER: we don't provide import and package statements in the code listings. Get the complete source code from the headfirstlabs web site. You'll find the URL on page xxxiii in the Intro.

TEST JAVA I/O DECORATOR MỚI CỦA BẠN

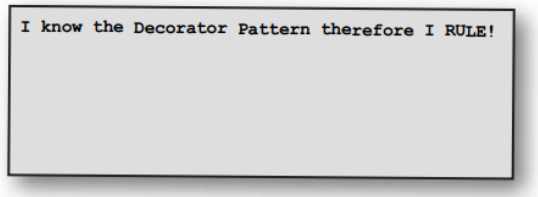
```
public class InputTest {
    public static void main(String[] args) throws IOException {
        int c;
        try {
            InputStream in =
                new LowerCaseInputStream(
                    new BufferedInputStream(
                        new FileInputStream("test.txt")));

            while((c = in.read()) >= 0) {
                System.out.print((char)c);
            }

            in.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Set up the FileInputStream and decorate it, first with a BufferedInputStream and then our brand new LowerCaseInputStream filter.

Just use the stream to read characters until the end of file and print as we go.



test.txt file

You need to make this file.

Give it a spin:

```
File Edit Window Help DecoratorsRule
% java InputTest
i know the decorator pattern therefore i rule!
%
```

Phỏng vấn tuần này: Lời thú tội của một Decorator Pattern

HeadFirst: Chào mừng Decorator Pattern. Chúng tôi nghe nói rằng bạn gần đây đã hơi thất vọng?

Decorator: Vâng, tôi biết thế giới coi tôi là mẫu thiết kế hấp dẫn, nhưng bạn biết đấy, tôi đã có những chia sẻ về vấn đề của mình đến mọi người.

HeadFirst: Có lẽ bạn nên chia sẻ một số rắc rối của bạn với chúng tôi?

Decorator: Chắc chắn rồi. Chà, bạn biết tôi đã có sức mạnh để thêm sự linh hoạt cho các thiết kế, điều đó là chắc chắn, nhưng tôi cũng có một mặt hạn chế. Bạn thấy đấy, đôi khi tôi có thể thêm rất nhiều lớp nhỏ vào một thiết kế và điều này đôi khi dẫn đến một thiết kế rất khó để người khác hiểu.

HeadFirst: Bạn có thể cho chúng tôi một ví dụ?

Decorator: Lấy các thư viện I/O Java. Đây là những thứ khó cho mọi người có thể hiểu ở lần đầu tiên. Nhưng nếu họ chỉ xem các lớp là một tập hợp các hàm bao quanh InputStream, cuộc sống sẽ dễ dàng hơn nhiều.

HeadFirst: Điều đó nghe có vẻ rất tệ. Bạn vẫn là một mô hình tuyệt vời, và cải thiện điều này chỉ là một vấn đề nhỏ, phải không?

Decorator: Còn nhiều hơn nữa, tôi sợ. Tôi đã gặp vấn đề: bạn thấy đấy, đôi khi người ta lấy một đoạn code dựa trên các lớp con và áp dụng vào các Decorator mà không suy nghĩ. Bây giờ, một điều tuyệt vời với tôi là bạn thường có thể thêm "trang trí trong suốt" và client không bao giờ phải biết nó. Nhưng như tôi đã nói, một số code phụ thuộc vào các lớp con và khi bạn bắt đầu đưa vào decorators, boom! Những điều tồi tệ sẽ xảy ra.

HeadFirst: Chà, tôi nghĩ mọi người đều hiểu rằng cần phải cẩn thận khi sử dụng Decorator, nhưng tôi không nghĩ rằng đây là một lý do để bạn coi thường bản thân.

Decorator: Tôi biết, tôi cố gắng không được. Tôi cũng có một vấn đề là áp dụng các Decorator có thể làm tăng độ phức tạp của code cần thiết để khởi tạo component. Khi bạn đã dùng Decorator, bạn không chỉ khởi tạo component mà còn bao bọc nó với những decorators.

HeadFirst: Tôi sẽ phỏng vấn các mẫu **Factory** và **Builder** vào tuần tới – Tôi nghe nói chúng có thể rất hữu ích với điều này?

Decorator: Điều đó là đúng; Tôi nên nói chuyện với những mẫu đó thường xuyên hơn.

HeadFirst: Chà, tất cả chúng ta đều nghĩ rằng bạn là một mẫu thiết kế tuyệt vời để tạo ra các thiết kế linh hoạt và đúng với Nguyên tắc Đóng mở, vì vậy hãy giữ nó và suy nghĩ tích cực!

Decorators: Tôi làm hết sức mình, cảm ơn bạn.

Tóm tắt

- Kế thừa là một hình thức mở rộng, nhưng không nhất thiết là cách tốt nhất để đạt được sự linh hoạt trong các thiết kế của chúng tôi.
- Trong các thiết kế của chúng tôi, chúng tôi nên cho phép hành vi được mở rộng mà không cần phải sửa đổi code hiện có.
- Composition và delegation thường có thể được sử dụng để thêm các hành vi mới *in runtime*.
- Decorator Pattern cung cấp một sự thay thế cho subclass để mở rộng hành vi.

- Decorator Pattern bao gồm một tập hợp các lớp trang trí được sử dụng để bọc các concrete component (concrete component kế thừa từ lớp component).
- Các lớp Decorator phản chiếu các loại thành phần mà nó trang trí. (Trên thực tế, chúng cùng loại với các component mà chúng trang trí, thông qua kế thừa hoặc implement interface.)
- Các Decorator thay đổi hành vi của các component của chúng bằng cách thêm chức năng mới trước và/hoặc sau (hoặc thậm chí thay thế) các cuộc gọi phương thức đến component.
- Bạn có thể bọc một component với bao nhiêu decorator tuỳ thích.
- Decorator thường transparent (trong suốt – nghĩa là client chỉ biết đến các lớp base class trừu tượng bên trên mà không cần biết đến các lớp con cụ thể) với client của component.
- Decorators có thể dẫn đến nhiều đối tượng nhỏ trong thiết kế của bạn và việc sử dụng quá mức có thể phức tạp.